Facultatea de Automatică şi Calculatoare, UPB
Departamentul de Calculatoare

Analiza algoritmilor

ŞL.dr.ing. Andrei Mogoş

2020-2021

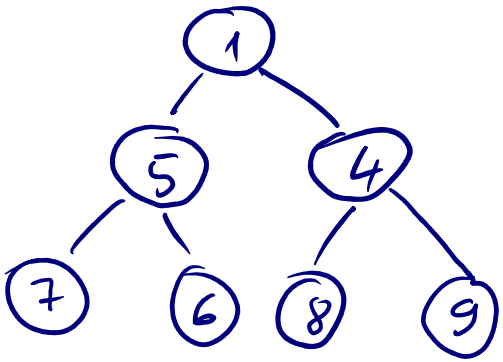
- Corectitudinea algoritmilor (3) -

Heap binar minimizant

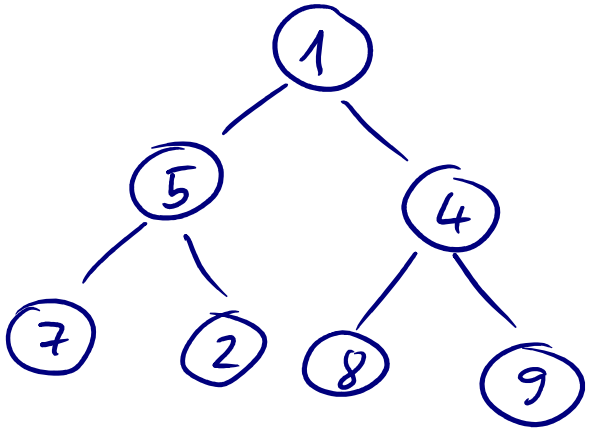
Def: Un **heap** (movilă) este o structură de date arborescentă performantă în raport cu operațiile de construcție, inserție, eliminare a elementelor și mai ales de selecție a elementului cu cheie minimă sau maximă existent în structură, operație care se realizează în timpul $O(1)$.

- Un heap este **minimizant** dacă operația de selecție returnează un element dintre cele cu cheie minimă în timpul $O(1)$.
- Un heap este **maximizant** dacă operația de selecție returnează un element dintre cele cu cheie maximă în timpul $O(1)$.
- În continuare considerăm heap minimizant.

Def: Un heap binar minimizant este un arbore binar astfel încât rădăcina conține elementul cel mai mic din arbore, iar subarborii stang și drept relativ la rădăcina heapului sunt tot heapuri.



heap binar minimizant



nu este heap binar minimizant

Specificare formală

- Elementele heapului sunt de tip T.

Valorile din T pot fi comparate utilizând operatorii: $<, >, \leq, \geq, =, \neq$.

$Bool = \{0, 1\}$

$\infty \notin T$, $T_\infty = T \cup \{\infty\}$

$\forall x \in T$ avem $x \neq \infty$, $x < \infty$ și $x \leq \infty$

$\infty = \infty$ și $\infty \leq \infty$

$$\min : T_\infty \times T_\infty \to T_\infty$$

$$\min(\infty, \infty) = \infty$$

$$\min(x, \infty) = \min(\infty, x) = x, \quad x \in T$$

$$\min(x, y) = \text{if } x \le y \text{ then } x \text{ else } y, \quad x, y \in T$$

TMset = multiseturi cu elemente de tip T. Un multiset este o "multime" în care elementele pot avea apariții multiple.

Operatori:

$\emptyset$: multisetul vid

$x + S$: inserția elementului x în multisetul S.
Dacă x apare în S de x_S ori atunci
x apare în $x + S$ de $x_S + 1$ ori

$S - x$: eliminarea elementului x din multisetul S.
Dacă x apare în S de x_S ori atunci
x apare în $S - x$ de $x_S - 1$ ori.

$S_1 \underline{\cup} S_2$: reuniunea multisetturilor S_1 și S_2

$x \# S$: numărul de apariții ale elementului x în S.

$TMset = \emptyset \mid T + TMset$

a) Constructori de bază

$\emptyset : \longrightarrow TMset$

$+ : T \times TMset \longrightarrow TMset$

b) Alți operatori

$- : TMset \times T \longrightarrow TMset$

$\underline{\cup} : TMset \times TMset \longrightarrow TMset$

$\# : T \times TMset \longrightarrow \mathbb{N}$

c) Axiome (descriu comportamentul celorlalți operatori în funcție de constructorii de bază)

$$\begin{cases} \phi - e = \phi \\ (e' + s) - e = \text{if } (e = e') \text{ then } s \\ \qquad\qquad\qquad\qquad \text{else } e' + (s - e) \end{cases}$$

$$\begin{cases} \phi \cup s = s \\ (e + s) \cup s' = e + (s \cup s') \end{cases}$$

$$\begin{cases} e \# \phi = 0 \\ e \# (e' + s) = \text{if } (e = e') \text{ then } 1 + (e \# s) \\ \qquad\qquad\qquad\qquad \text{else } (e \# s) \end{cases}$$

Obs: Pe baza axiomelor, orice formulă în care apar operatorii tipului poate fi redusă la o formulă în care apar doar constructorii de bază.

Ex:

$$(1 + \phi) \underline{\cup} ((1 + (2 + (2 + \phi))) - 2) \overset{\text{"}-\text{"}}{=}$$

$$= (1 + \phi) \underline{\cup} (1 + ((2 + (2 + \phi)) - 2)) \overset{\text{"}-\text{"}}{=}$$

$$= (1 + \phi) \underline{\cup} (1 + (2 + \phi)) \overset{\text{"}\cup\text{"}}{=}$$

$$= (1 + (1 + (2 + \phi))).$$

Tipul TArb; Multimile THeap și TArh

$TArb = A_\phi \mid arb(T, TArb, TArb)$

$THeap = \{A \in TArb \mid e_heap(A)\}$

$TArh = \{A \in TArb \mid A \neq A_\phi \Rightarrow e_heap(S(A)) \text{ și } e_heap(D(A))\}$

a) Constructori de bază

$A_\phi :\rightarrow TArb$ // arborele nul

$arb : T \times TArb \times TArb \rightarrow TArb$
// rădăcină, subarbore stang, subarbore drept => nou arbore

b) Alti operatori

$s : TArb \setminus \{A_\phi\} \to TArb$ // subarborele stâng

$d : TArb \setminus \{A_\phi\} \to TArb$ // subarborele drept

$top : TArb \to T_\infty$ // rădăcina

$elm : TArb \to TMset$ // multisetul cu nodurile arborelui

$e_heap : TArb \to Bool$

$cons : TArb \to THeap$

c) Axiome

$$S(arb(x, A_s, A_d)) = A_s$$

$$D(arb(x, A_s, A_d)) = A_d$$

$$\begin{cases} top(A_\emptyset) = \infty \\ top(arb(x, A_s, A_d)) = x \end{cases}$$

$$\begin{cases} elm(A_\emptyset) = \emptyset \\ elm(arb(x, A_s, A_d)) = x + (elm(A_s) \cup elm(A_d)) \end{cases}$$

$$\begin{cases} e_heap(A_\emptyset) = 1 \\ e_heap(arb(x, A_s, A_d)) = (x \leq min(top(A_s), top(A_d))) \\ \qquad \text{si } e_heap(A_s) \text{ si } e_heap(A_d) \end{cases}$$

$$
\begin{cases}
\mathrm{cern}(A_\phi) = A_\phi \\
\mathrm{cern}(\mathrm{anb}(x, A_s, A_d)) = \\
\quad \text{if } x \le \min(\mathrm{top}(A_s), \mathrm{top}(A_d)) \text{ then } \mathrm{anb}(x, A_s, A_d) \\
\quad \text{else} \\
\quad\quad \text{if } \mathrm{top}(A_s) \le \mathrm{top}(A_d) \text{ then} \\
\quad\quad\quad \mathrm{anb}(\mathrm{top}(A_s), \mathrm{cern}(\mathrm{anb}(x, s(A_s), D(A_s))), A_d) \\
\quad\quad \text{else} \\
\quad\quad\quad \mathrm{anb}(\mathrm{top}(A_d), A_s, \mathrm{cern}(\mathrm{anb}(x, s(A_d), D(A_d))))
\end{cases}
$$

Prop 1: $\forall A \in TArb$ avem $elm(cem(A)) = elm(A)$

Obs: Această proprietate se poate demonstra prin inducție structurală.

Prop 2: $\forall A \in THeap$ avem $P(A)$ adevărat, unde $P(A)$ este definită astfel:

$P(A) = def$ $\forall e \in T$ cu prop. că $e \# elm(A) > 0$ avem $top(A) \leq e$

□ Demonstrăm prin inducție structurală

° Prop 2.

Demo

Caz de bază: $A = A_\emptyset$

$elm(A) = elm(A_\emptyset) \overset{elm}{=} \emptyset \Rightarrow e \# elm(A) = 0$

$\Rightarrow P(A)$ adevărată (implicit)

Pas de inducție: $A = arb(x, A_s, A_d)$

Ipoteză de inducție: $P(A_s)$, $P(A_d)$ adevărate

($A_s, A_d \in THeap$)

De asemenea:

$A \in THeap \Rightarrow \begin{cases} x \leq top(A_s) \\ x \leq top(A_d) \end{cases}$ (*)

($x = top(A)$)

Arătăm că $P(A)$ adevărat

Din ipoteza de inducție avem:

$\forall e \in T$ cu prop. că $e \# elm(A_s) > 0$ avem $top(A_s) \leq e$

$\forall e \in T$ cu prop. că $e \# elm(A_d) > 0$ avem $top(A_d) \leq e$

$\Rightarrow$

(*)

$\Rightarrow \forall e \in T$ cu prop. că

$e \# (x + (elm(A_s) \cup elm(A_d))) > 0$

avem $x \leq e$

$\Rightarrow \forall e \in T$ cu prop. că $e \# elm(A) > 0$ avem $x \leq e$.

$\Rightarrow P(A)$ adevărat

Caz de bază verificat

Pas de inducție verificat $\Rightarrow$

$\Rightarrow \forall A \in THeap$ avem $P(A)$ adevărat

Prop 3 : $\forall A \in TArb$ avem $e_heap(cem(A)) = 1$

(adică $cem(A) \in THeap$)

Demonstrație prin inducție structurală

Caz de bază : $A = A_\emptyset$

$e_heap(cem(A)) = e_heap(cem(A_\emptyset)) \overset{cem}{=}$

$= e_heap(A_\emptyset) \overset{e_heap}{=} 1$

Pas de inducție : $A = arb(x, A_s, A_d)$

$(A_s, A_d \in THeap)$

Ipoteza de inducție

$e_heap(cem(A_s)) = 1$

$e_heap(cem(A_d)) = 1$

și proprietatea este adevărată pentru toți arborii de aceeași dimensiune cu A_s și A_d.

Arătăm că : $e_heap(cem(A)) = 1$.

$y = \min(top(A_s), top(A_d))$

Caz 1: $x \leq y$

$e_heap(cons(A)) = e_heap(cons(arb(x, A_s, A_d))) =$

$cons = e_heap(arb(x, A_s, A_d)) \overset{e_heap}{=} x \leq y$, si

$e_heap(A_s)$ si $e_heap(A_d) = 1$

Caz 2: $x > y$ si $top(A_s) \leq top(A_d)$

$\Rightarrow y = top(A_s)$

$x \in T, x > y \Rightarrow y \neq \infty \overset{top}{\Rightarrow} A_s \neq A_\emptyset$

$\Rightarrow \exists S(A_s), D(A_s)$.

Fie $A_s' = cons(arb(x, S(A_s), D(A_s)))$.

$l_heap(cons(A)) = l_heap(cons(arb(x, A_s, A_d))) =$

$\overset{cons}{=} l_heap(arb(y, A_s', A_d)) \overset{l_heap}{=}$

$= y \leq min(top(A_s'), top(A_d))$, și $\underbrace{l_heap(A_s')}_{\text{adevărat din ipoteza de inducție}}$

, și $\underbrace{l_heap(A_d)}_{\text{adevărat pt că } A_d \in THeap} =$

$= y \leq top(A_s')$ (1)

$A_s \in THeap \overset{Prop\ 2}{\Rightarrow}$

$\Rightarrow \forall e \in T$ cu proprietatea că $e\#elem(A_s) > 0$ avem $y \leq e$

$x > y \Rightarrow \forall e \in T$ cu proprietatea că $e\#(x + (elem(A_s) - y)) > 0$ avem $y \leq e$

$elem(A'_s) \overset{Prop\ 1}{=} elem(arb(x, S(A_s), D(A_s))) =$

$\overset{elem}{=} x + (elem(S(A_s)) \overset{\cup}{-} elem(D(A_s))) =$

$= x + (elem(A_s) - y)$

$\Rightarrow \forall e \in T$ un $prop.$ că $e \# elm(A_s') > 0$

avem $y \leq e$

$top(A_s') \in elm(A_s')$ $\Rightarrow$

$\Rightarrow y \leq top(A_s')\ (2)$

Din (1) și (2) $\Rightarrow e.heap(arm(A)) = 1.$

Caz 3 : $x > y$ și $top(A_s) > top(A_d)$

Analog ca cazul 2.

Caz de bază verificat

Pas de inducție verificat $\Big| \Rightarrow$

$\Rightarrow \forall A \in TArb$ avem e_heap(cem(A)) = 1.

Obs: Când aplicăm inducția structurală este suficient să folosim doar constructorii de bază (deoarece folosind constructorii de bază se poate genera orice element al tipului de date).